

From Sequential Models to Transformers, BERT, and Large Language Models

A Comprehensive Mathematical and Architectural Reference

Chandra Prakash Bathula

April 20, 2026

Contents

1	Introduction: The Sequence Modelling Problem	3
2	Classical Sequential Models	3
2.1	Vanilla Recurrent Neural Network (RNN)	3
2.1.1	Backpropagation Through Time (BPTT) and Vanishing Gradients	4
2.2	Long Short-Term Memory (LSTM)	4
2.3	Gated Recurrent Unit (GRU)	5
2.4	Fundamental Limitations of Recurrent Models	6
3	The Attention Mechanism	6
3.1	Motivation: Encoder–Decoder Attention	6
3.2	Scaled Dot-Product Attention	6
3.3	Self-Attention	6
3.3.1	Worked Numeric Example	7
3.4	Multi-Head Attention (MHA)	7
3.5	Positional Encoding	8
4	The Transformer Architecture	9
4.1	Transformer Encoder Block	9
4.2	Layer Normalization	9
4.3	Transformer Decoder Block	9
4.4	Full Encoder–Decoder Transformer	10
4.5	Three Transformer Architectural Families	10
4.6	Encoder and Decoder Dataflow in Detail	10
4.7	Cross-Attention in Detail	11
4.8	Complexity Analysis	11
5	BERT: Bidirectional Encoder Representations from Transformers	11
5.1	Overview	11
5.2	Input Representation	14
5.3	BERT Encoder Stack	14
5.4	Pre-Training Objectives	15
5.4.1	Masked Language Modelling (MLM)	15
5.4.2	Next Sentence Prediction (NSP)	15
5.5	Fine-Tuning for Downstream Tasks	16
5.5.1	Sequence Classification	16

5.5.2	Token-Level Classification (NER)	16
5.5.3	Extractive Question Answering	16
5.6	BERT Variants and Successors	17
6	Autoregressive Language Models: GPT Family	17
6.1	Causal (Decoder-Only) Transformer	17
6.2	Scaling Laws	17
7	Large Language Models (LLMs)	18
7.1	Architecture Patterns in Modern LLMs	18
7.2	Instruction Tuning and RLHF	19
7.3	Key Modern LLMs	19
7.4	Mixture of Experts (MoE)	19
8	Efficient Attention Mechanisms	20
8.1	FlashAttention	20
8.2	Linear Attention	21
8.3	Sparse and Local Attention	21
9	Comparing Architectures: A Unified View	21
10	Tokenisation	22
10.1	Embedding Lookup and Special Tokens	22
10.2	Byte-Pair Encoding (BPE)	22
10.3	WordPiece	23
10.4	Unigram Language Model	23
10.5	Comparing Subword Tokenisers and Attention Masks	23
11	Training Techniques for LLMs	24
11.1	Mixed-Precision Training	24
11.2	Gradient Checkpointing	24
11.3	Parameter-Efficient Fine-Tuning (PEFT)	24
11.4	Data Parallelism, Tensor Parallelism, Pipeline Parallelism	24
12	Evaluation	24
12.1	Perplexity	24
12.2	Benchmark Suites	25
13	Conclusion	25

1 Introduction: The Sequence Modelling Problem

Many real-world problems are intrinsically sequential. Formal definitions help fix notation.

What Is a Sequence?

A *sequence* of length T over a domain $\mathcal{X}$ is an ordered tuple $\mathbf{x} = (x_1, x_2, \dots, x_T)$ with $x_t \in \mathcal{X}$ for all t . In NLP, $\mathcal{X}$ is a vocabulary $\mathcal{V}$ and each $x_t \in \{1, \dots, |\mathcal{V}|\}$ is a token index.

Core learning objectives for sequence models:

1. **Sequence-to-label:** $f : \mathcal{X}^T \rightarrow \mathcal{Y}$ — e.g., sentiment classification.
2. **Sequence-to-sequence:** $f : \mathcal{X}^T \rightarrow \mathcal{Y}^S$ — e.g., machine translation.
3. **Token labelling:** assign a label to every position — e.g., named entity recognition.
4. **Language modelling:** estimate $p(x_1, \dots, x_T) = \prod_t p(x_t | x_{<t})$.

The key challenge is capturing *long-range dependencies*: the meaning of a token at position t often depends critically on tokens far away. The evolution of architectures is largely the story of solving this challenge.

2 Classical Sequential Models

2.1 Vanilla Recurrent Neural Network (RNN)

Let $x_t \in \mathbb{R}^d$ be the embedding of the t -th token. An RNN maintains a hidden state $h_t \in \mathbb{R}^H$:

RNN Equations

$$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b_h), \quad (1)$$

$$\hat{y}_t = \text{softmax}(W_{hy} h_t + b_y). \quad (2)$$

where $W_{xh} \in \mathbb{R}^{H \times d}$, $W_{hh} \in \mathbb{R}^{H \times H}$, $W_{hy} \in \mathbb{R}^{O \times H}$, $b_h \in \mathbb{R}^H$, $b_y \in \mathbb{R}^O$.

The same weight matrices are *shared* across all time steps — this enables generalisation to sequences of any length.

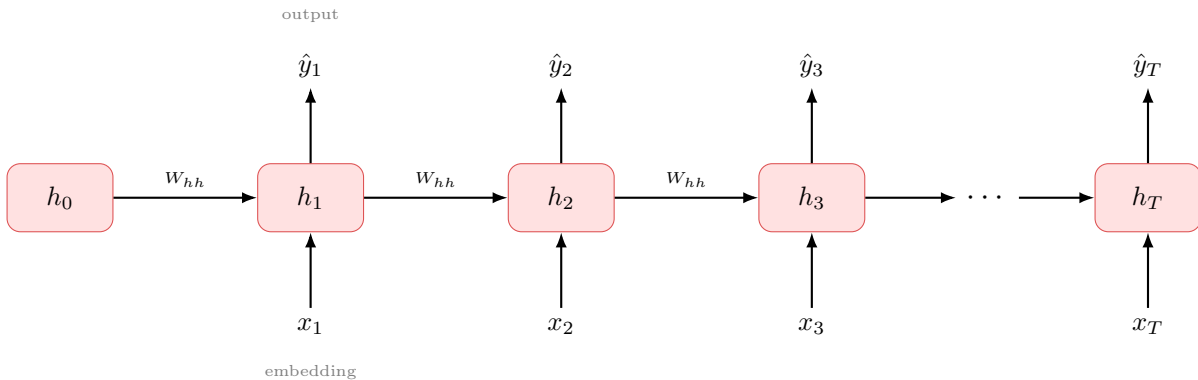


Figure 1: Unrolled vanilla RNN. The same weight matrices W_{xh} , W_{hh} , W_{hy} are reused at every step. The hidden state h_t summarises all past inputs.

2.1.1 Backpropagation Through Time (BPTT) and Vanishing Gradients

Training uses BPTT. The gradient of the loss $\mathcal{L}$ with respect to W_{hh} involves products of Jacobians:

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^T \frac{\partial \mathcal{L}_t}{\partial h_t} \left(\prod_{s=k+1}^t \frac{\partial h_s}{\partial h_{s-1}} \right) \frac{\partial h_k}{\partial W_{hh}}, \quad \text{where} \quad \frac{\partial h_s}{\partial h_{s-1}} = \text{diag}(1 - \tanh^2(a_s)) W_{hh}. \quad (3)$$

Let $\rho = \|W_{hh}\|$ and the diagonal entries of the tanh derivative be bounded by 1. Then the norm of the product over k steps satisfies:

$$\left\| \prod_{s=k+1}^t \frac{\partial h_s}{\partial h_{s-1}} \right\| \leq \rho^{t-k}. \quad (4)$$

Key Insight: Vanishing / Exploding Gradients

If $\rho < 1$, gradients vanish exponentially as $t - k$ grows — the network forgets distant past.
If $\rho > 1$, gradients explode. Both make training unstable for long sequences.

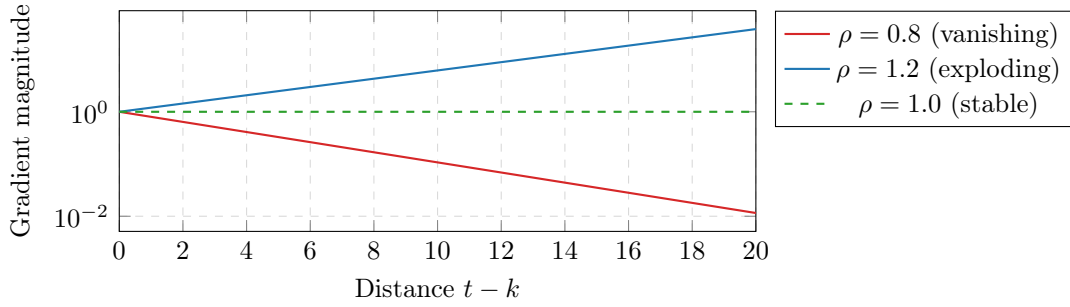


Figure 2: Gradient magnitude vs. distance for different spectral radii ρ of W_{hh} .

2.2 Long Short-Term Memory (LSTM)

Hochreiter & Schmidhuber (1997) introduced the LSTM to address vanishing gradients through explicit *gating* and a separate *cell state* c_t .

LSTM Equations

$$f_t = \sigma(W_f[h_{t-1}; x_t] + b_f) \quad (\text{forget gate}), \quad (5)$$

$$i_t = \sigma(W_i[h_{t-1}; x_t] + b_i) \quad (\text{input gate}), \quad (6)$$

$$\tilde{c}_t = \tanh(W_c[h_{t-1}; x_t] + b_c) \quad (\text{candidate cell}), \quad (7)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (\text{cell state update}), \quad (8)$$

$$o_t = \sigma(W_o[h_{t-1}; x_t] + b_o) \quad (\text{output gate}), \quad (9)$$

$$h_t = o_t \odot \tanh(c_t) \quad (\text{hidden state}). \quad (10)$$

Here $[a; b]$ is concatenation, σ is the logistic sigmoid, $\odot$ is elementwise product.

Why does the LSTM mitigate vanishing gradients? The cell-state gradient:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t.$$

When $f_t \approx 1$ (forget gate open), the gradient flows back unimpeded. The LSTM can maintain near-constant gradient magnitude over many steps.

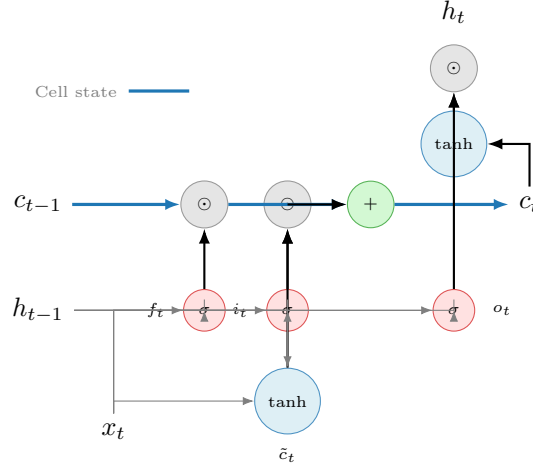


Figure 3: LSTM cell. The horizontal line carries the cell state c_t , modified only by element-wise operations — this "highway" enables long-range gradient flow.

2.3 Gated Recurrent Unit (GRU)

The GRU (Cho et al., 2014) simplifies the LSTM by merging the cell state and hidden state.

GRU Equations

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (\text{update gate}), \quad (11)$$

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (\text{reset gate}), \quad (12)$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \quad (\text{candidate}), \quad (13)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \quad (\text{output}). \quad (14)$$

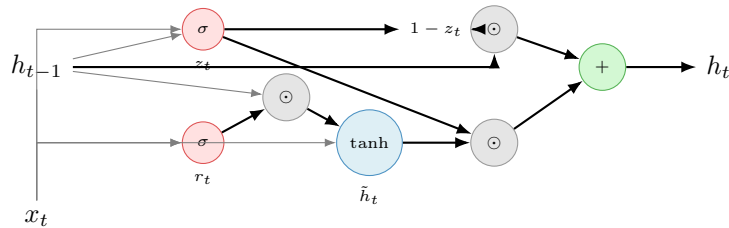


Figure 4: GRU cell. Two gates (update z_t , reset r_t) control information flow without a separate cell state.

Limitation	Root Cause	Consequence
Sequential computation	h_t depends on h_{t-1}	Cannot parallelise over time
Long path length	$O(T)$ steps to connect positions	Hard long-range dependencies
Fixed-size bottleneck	h_T compresses full sequence	Information loss at large T
Bidirectionality cost	Requires two passes	Doubles compute; still recurrent

Table 1: Core limitations of recurrent architectures.

2.4 Fundamental Limitations of Recurrent Models

3 The Attention Mechanism

3.1 Motivation: Encoder–Decoder Attention

Bahdanau et al. (2015) added an *attention* module to RNN-based seq2seq models. Instead of compressing the full source into one vector, each decoder step queries all encoder hidden states. This was the seed idea that grew into the Transformer.

3.2 Scaled Dot-Product Attention

Core Definition

Given a query $q \in \mathbb{R}^{d_k}$, a set of keys $K \in \mathbb{R}^{n \times d_k}$, and values $V \in \mathbb{R}^{n \times d_v}$:

$$\text{Attention}(q, K, V) = \sum_{j=1}^n \frac{\exp(q^\top k_j / \sqrt{d_k})}{\underbrace{\sum_{\ell} \exp(q^\top k_\ell / \sqrt{d_k})}_{\alpha_j}} v_j.$$

In matrix form, with queries $Q \in \mathbb{R}^{T \times d_k}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \in \mathbb{R}^{T \times d_v}.$$

Why divide by $\sqrt{d_k}$? If q and k have i.i.d. zero-mean, unit-variance entries, then $q^\top k = \sum_{i=1}^{d_k} q_i k_i$ has variance d_k . Dividing by $\sqrt{d_k}$ restores unit variance, preventing the dot products from entering saturation regions of softmax.

3.3 Self-Attention

In *self-attention* all three matrices Q, K, V are derived from the *same* input sequence $X \in \mathbb{R}^{T \times d_{\text{model}}}$ via learnable projections:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V, \tag{15}$$

where $W^Q, W^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$.

The output $Z_i = \sum_j A_{ij} v_j$ is a context-aware re-encoding of token i using information from *all* other tokens simultaneously.

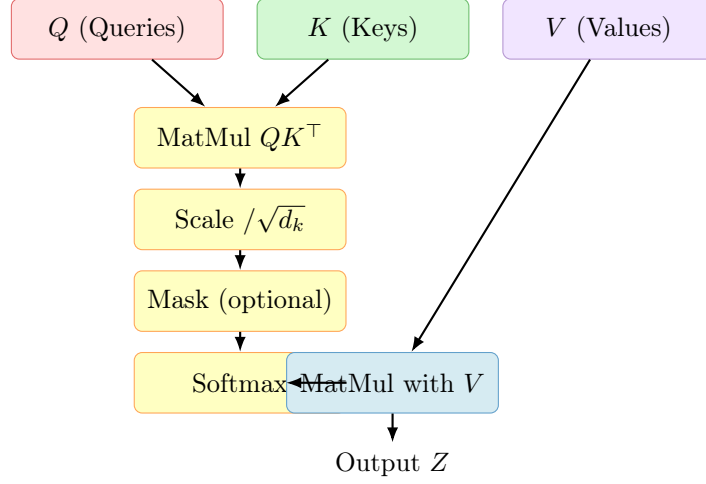


Figure 5: Scaled dot-product attention computation graph.

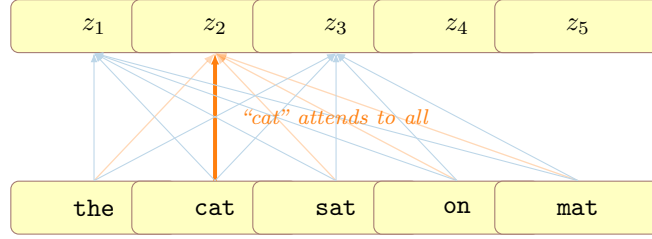


Figure 6: Self-attention: each output z_i is a weighted blend of all input representations. The weights are learned and input-dependent.

3.3.1 Worked Numeric Example

Let $T = 3$, $d_k = 2$, and suppose after projection:

$$Q = K = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}.$$

Then

$$S = \frac{QK^\top}{\sqrt{2}} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}.$$

Row-1 after softmax: $A_{1,:} = \text{softmax}([1/\sqrt{2}, 0, 1/\sqrt{2}]) \approx [0.422, 0.155, 0.422]$.

Token 1 attends equally to tokens 1 and 3, weakly to token 2.

3.4 Multi-Head Attention (MHA)

Multi-Head Attention

Run h attention heads in parallel, each with its own projections:

$$\text{head}_i = \text{Attention}(XW_i^Q, XW_i^K, XW_i^V), \quad i = 1, \dots, h,$$

then concatenate and project:

$$\text{MHA}(X) = [\text{head}_1; \dots; \text{head}_h] W^O,$$

where $W_i^Q, W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$, $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$. Typically $d_k = d_v =$

$$d_{\text{model}}/h.$$

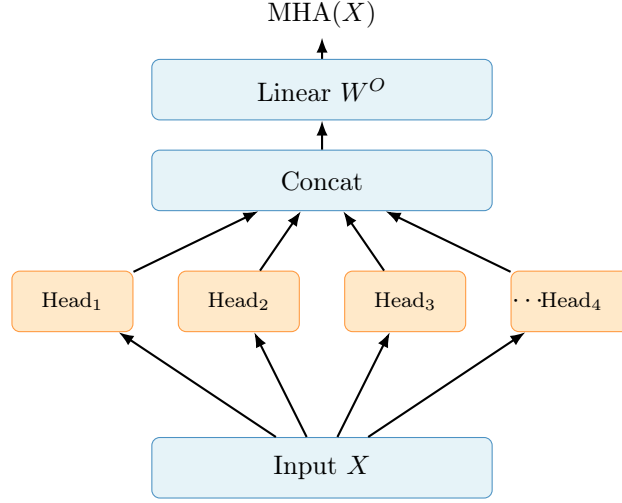


Figure 7: Multi-head attention: parallel attention heads capture different relationship patterns, then combine through concatenation and linear projection.

3.5 Positional Encoding

Self-attention is *permutation equivariant*: it treats the input as a set, not a sequence. To inject order, we add a positional encoding $\text{PE} \in \mathbb{R}^{T \times d_{\text{model}}}$ to the token embeddings:

$$E_t = \text{Embed}(x_t) + \text{PE}(t).$$

Sinusoidal Positional Encoding (Vaswani et al., 2017).

$$\text{PE}(t, 2i) = \sin\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right), \quad (16)$$

$$\text{PE}(t, 2i + 1) = \cos\left(\frac{t}{10000^{2i/d_{\text{model}}}}\right). \quad (17)$$

Properties: fixed (no extra parameters), extends to unseen lengths, encodes relative distance since $\text{PE}(t + k)$ can be expressed as a linear function of $\text{PE}(t)$.

Learned Positional Embeddings. Replace the formula with a trainable matrix $P \in \mathbb{R}^{T_{\text{max}} \times d_{\text{model}}}$, where P_t is looked up by position. Used in BERT and GPT.

Rotary Positional Embedding (RoPE). Used in LLaMA, PaLM, and many modern LLMs:

$$q'_i = q_i e^{im\theta}, \quad k'_j = k_j e^{-im\theta},$$

so $q'_i{}^\top k'_j = q_i{}^\top k_j \cos((i-j)\theta) + \dots$, encoding only *relative* position. Extrapolates well to sequences longer than those seen during training.

4 The Transformer Architecture

4.1 Transformer Encoder Block

Each encoder block consists of two sub-layers with residual connections and layer norm:

Encoder Layer

$$\tilde{H}^{(l)} = \text{LayerNorm}(H^{(l-1)} + \text{MHA}(H^{(l-1)})), \quad (18)$$

$$H^{(l)} = \text{LayerNorm}(\tilde{H}^{(l)} + \text{FFN}(\tilde{H}^{(l)})), \quad (19)$$

where $\text{FFN}(h) = \text{GELU}(hW_1 + b_1)W_2 + b_2$, with $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$, $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$, typically $d_{\text{ff}} = 4d_{\text{model}}$.

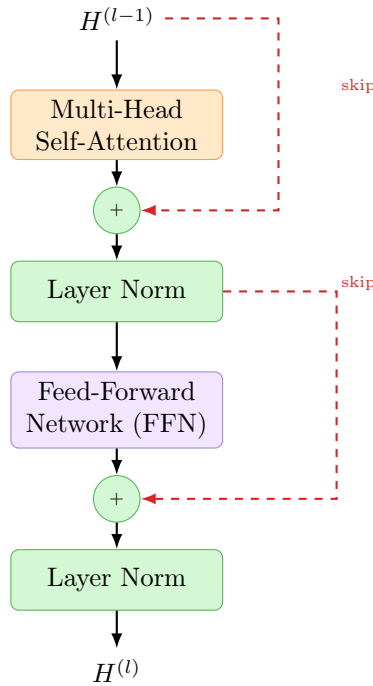


Figure 8: Single Transformer encoder layer. Residual connections (dashed red) allow gradient flow regardless of depth.

4.2 Layer Normalization

For a vector $h \in \mathbb{R}^H$:

$$\text{LayerNorm}(h) = \gamma \odot \frac{h - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta, \quad \mu = \frac{1}{H} \sum_i h_i, \quad \sigma^2 = \frac{1}{H} \sum_i (h_i - \mu)^2,$$

where $\gamma, \beta \in \mathbb{R}^H$ are learnable and $\epsilon > 0$ prevents division by zero. Unlike batch norm, layer norm operates over the feature dimension — stable for variable-length sequences and small batch sizes.

4.3 Transformer Decoder Block

The decoder adds a third sub-layer — *cross-attention* — that attends to the encoder output.

Decoder Layer (three sub-layers)

$$\tilde{H}_{\text{dec}}^{(l)} = \text{LN}(H_{\text{dec}}^{(l-1)} + \text{MaskedMHA}(H_{\text{dec}}^{(l-1)})), \quad (20)$$

$$\hat{H}_{\text{dec}}^{(l)} = \text{LN}(\tilde{H}_{\text{dec}}^{(l)} + \text{CrossAttn}(\tilde{H}_{\text{dec}}^{(l)}, H_{\text{enc}}^{(L)})), \quad (21)$$

$$H_{\text{dec}}^{(l)} = \text{LN}(\hat{H}_{\text{dec}}^{(l)} + \text{FFN}(\hat{H}_{\text{dec}}^{(l)})). \quad (22)$$

Causal Mask. To prevent the decoder from attending to future tokens:

$$M_{ij} = \begin{cases} 0 & j \leq i \\ -\infty & j > i \end{cases}, \quad \text{MaskedAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V.$$

4.4 Full Encoder–Decoder Transformer

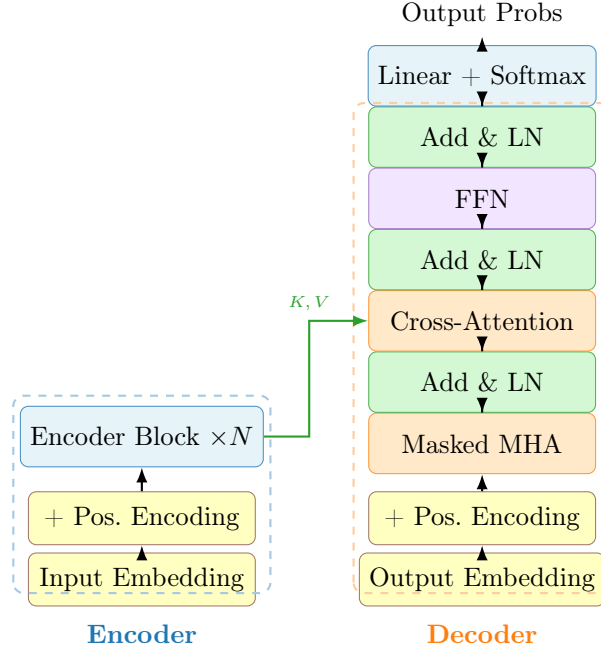


Figure 9: Full encoder–decoder Transformer (“Attention Is All You Need”, Vaswani et al., 2017). The cross-attention sublayer in the decoder queries the encoder’s final representations.

4.5 Three Transformer Architectural Families

The same attention building blocks are arranged differently depending on the task.

4.6 Encoder and Decoder Dataflow in Detail

Let the source sequence length be T_x , target length be T_y , model width be d_{model} , number of heads be h , and per-head width be $d_k = d_v = d_{\text{model}}/h$.

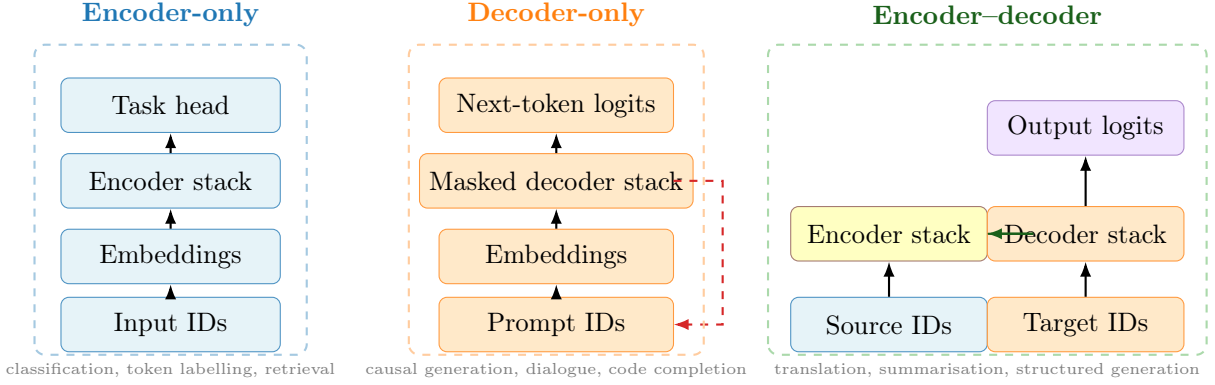


Figure 10: Transformer families differ mainly in which stacks are present and whether self-attention is bidirectional or causal.

4.7 Cross-Attention in Detail

Cross-attention differs from self-attention only in the source of the projections:

$$Q = H_{\text{dec}} W_Q \in \mathbb{R}^{T_y \times d_k}, \quad (23)$$

$$K = H_{\text{enc}} W_K \in \mathbb{R}^{T_x \times d_k}, \quad (24)$$

$$V = H_{\text{enc}} W_V \in \mathbb{R}^{T_x \times d_v}, \quad (25)$$

$$A = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) \in \mathbb{R}^{T_y \times T_x}, \quad (26)$$

$$\text{CrossAttn}(H_{\text{dec}}, H_{\text{enc}}) = AV. \quad (27)$$

Each row of A is an alignment distribution over source positions for one target token.

4.8 Complexity Analysis

Operation	Compute	Sequential	Max Path Length
RNN / LSTM	$O(Td^2)$	$O(T)$	$O(T)$
Self-Attention	$O(T^2d)$	$O(1)$	$O(1)$
CNN (kernel k)	$O(kTd^2)$	$O(1)$	$O(\log_k T)$
Local Attn (window w)	$O(wTd)$	$O(1)$	$O(T/w)$

Table 2: Complexity comparison per layer. T = sequence length, d = hidden dimension. Self-attention achieves $O(1)$ maximum path length at the cost of $O(T^2)$ compute.

5 BERT: Bidirectional Encoder Representations from Transformers

5.1 Overview

BERT (Devlin et al., 2019) is a *pre-trained encoder-only* transformer trained with self-supervised objectives on large unlabelled corpora. The key insight:

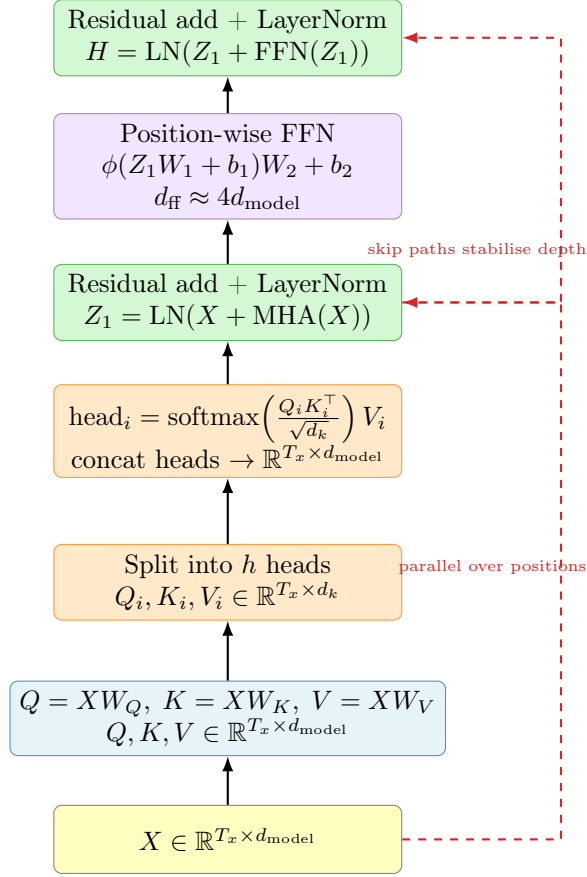


Figure 11: Internal encoder dataflow with tensor shapes. Every source position can attend to every other source position.

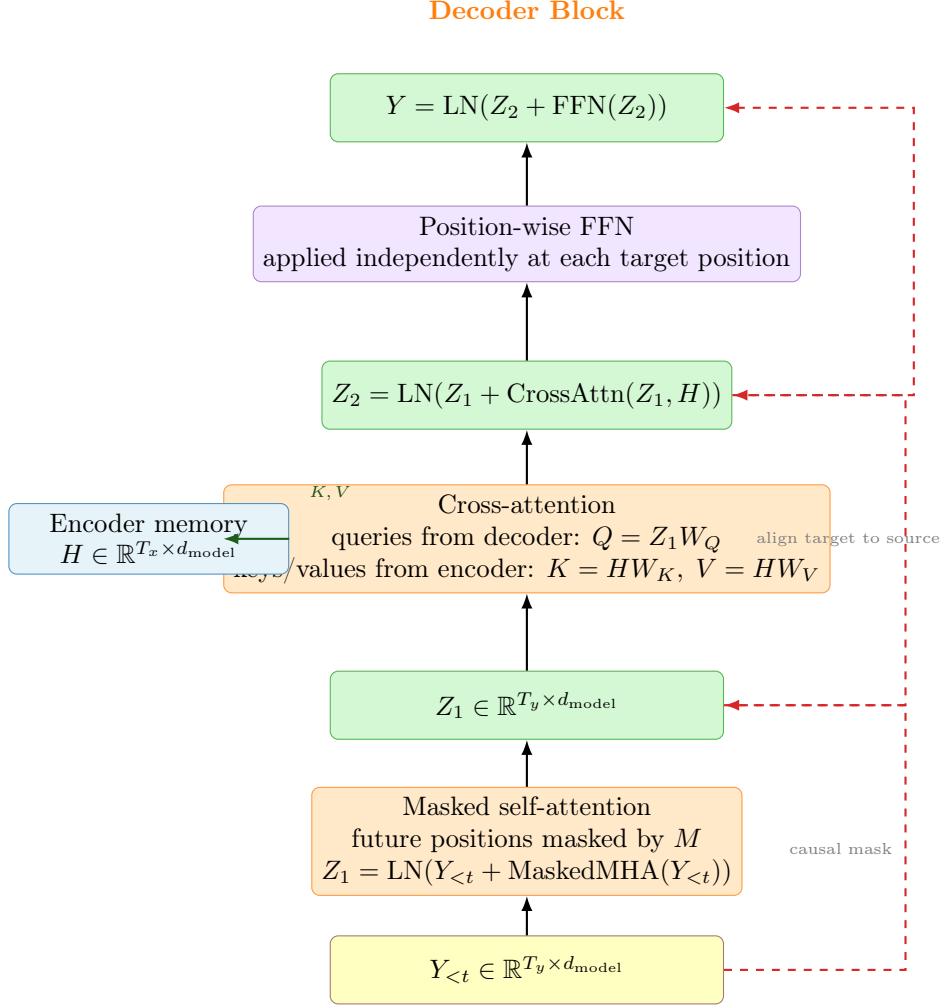


Figure 12: Internal decoder dataflow. The decoder first models target-side context causally, then aligns each target position to encoder representations through cross-attention.

Example: French → English alignment

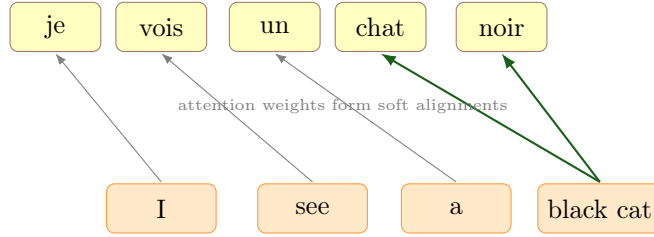


Figure 13: Cross-attention often behaves like a soft alignment matrix between target and source tokens. Multi-head attention lets different heads capture different alignment patterns.

Key Insight: BERT's Core Idea

Use a bidirectional transformer encoder — attending to *both* left and right context simultaneously — trained with masked language modelling. Fine-tune the entire model (plus a small task head) for each downstream task.

5.2 Input Representation

Every input is structured as:

$$[\text{CLS}] \ w_1 \ \cdots \ w_m \ [\text{SEP}] \ w_{m+1} \ \cdots \ w_n \ [\text{SEP}].$$

Three embeddings are summed at each position t :

$$E_t = E_t^{\text{tok}} + E_t^{\text{seg}} + E_t^{\text{pos}}. \quad (28)$$

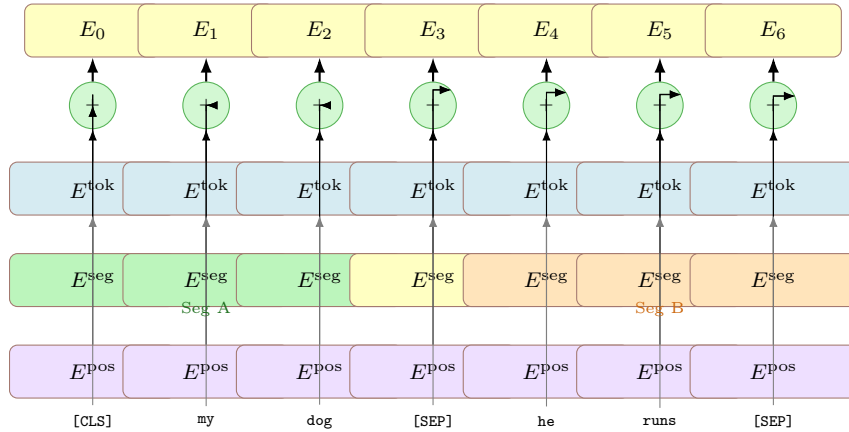


Figure 14: BERT input: token, segment, and positional embeddings are summed element-wise at each position.

5.3 BERT Encoder Stack

BERT-Base: $L = 12$ layers, $H = 768$, $A = 12$ attention heads, $\sim 110\text{M}$ parameters.

BERT-Large: $L = 24$ layers, $H = 1024$, $A = 16$ heads, $\sim 340\text{M}$ parameters.

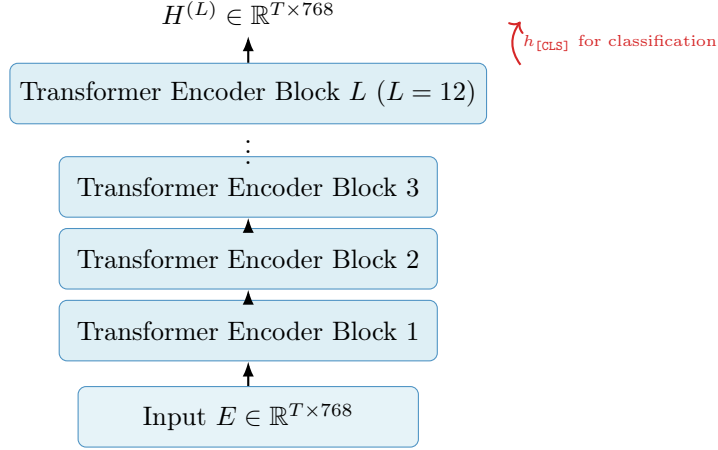


Figure 15: BERT encoder: 12 identical blocks transform the input into context-aware representations.

5.4 Pre-Training Objectives

5.4.1 Masked Language Modelling (MLM)

1. Sample 15% of token positions uniformly: set $\mathcal{M}$.
2. For each $t \in \mathcal{M}$:
 - With prob. 0.80: replace w_t with [MASK].
 - With prob. 0.10: replace w_t with a random token.
 - With prob. 0.10: keep w_t unchanged.
3. Predict original token at masked positions.

MLM Loss

$$\mathcal{L}_{\text{MLM}}(\theta) = - \sum_{t \in \mathcal{M}} \log p_{\theta}(w_t \mid X_{\text{masked}}),$$

where $p_{\theta}(w \mid X_{\text{masked}}) = \text{softmax}(W_{\text{vocab}} h_t^{(L)} + b)_w$.

The 10%/10% perturbation prevents the model from learning simply that [MASK] tokens should be predicted — it must remain robust to the full input distribution.

5.4.2 Next Sentence Prediction (NSP)

1. With prob. 0.50: B is the sentence immediately following A in the corpus (ISNEXT).
2. With prob. 0.50: B is a random sentence from a different document (NOTNEXT).

NSP Loss

$$\mathcal{L}_{\text{NSP}}(\theta) = -[y \log p(\text{ISNEXT}) + (1-y) \log p(\text{NOTNEXT})], \quad p = \text{softmax}(W_{\text{NSP}} h_{[\text{CLS}]}^{(L)} + b_{\text{NSP}}).$$

Total pre-training loss.

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{MLM}}(\theta) + \mathcal{L}_{\text{NSP}}(\theta).$$

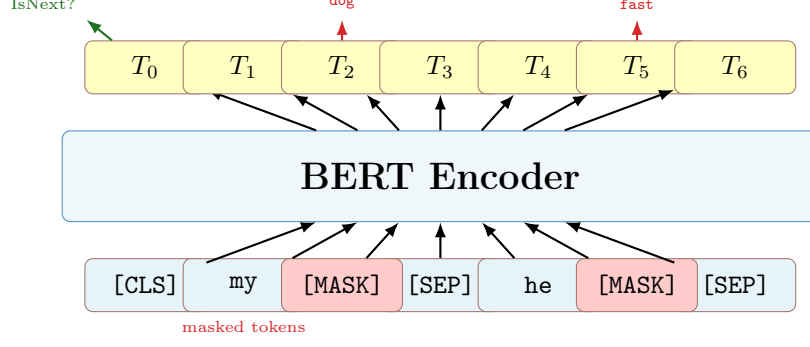


Figure 16: BERT pre-training: MLM predicts masked tokens (red); NSP predicts whether sentence B follows sentence A using the [CLS] representation.

5.5 Fine-Tuning for Downstream Tasks

5.5.1 Sequence Classification

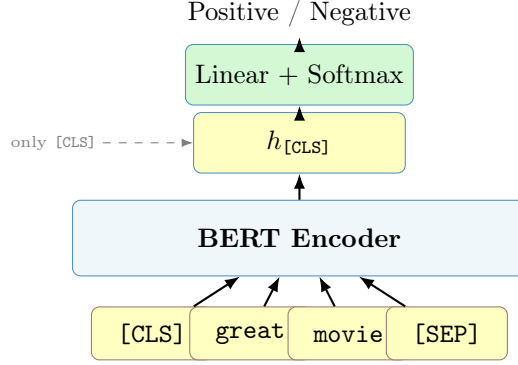


Figure 17: BERT fine-tuned for sentiment classification. Only the [CLS] representation feeds into a linear classifier head.

Mathematically:

$$\hat{y} = \text{softmax}(W_c h_{[\text{CLS}]}^{(L)} + b_c), \quad \mathcal{L} = - \sum_c y_c \log \hat{y}_c.$$

5.5.2 Token-Level Classification (NER)

For each position t :

$$\hat{y}_t = \text{softmax}(W_{\text{tok}} h_t^{(L)} + b_{\text{tok}}).$$

Labels: e.g. B-PER, I-PER, O, B-LOC, ... in BIO scheme.

5.5.3 Extractive Question Answering

Input: [CLS] question [SEP] context [SEP].

Learn two vectors $w_s, w_e \in \mathbb{R}^H$:

$$s_t = w_s^\top h_t, \quad p_{\text{start}}(t) = \frac{e^{s_t}}{\sum_j e^{s_j}}, \quad (29)$$

$$e_t = w_e^\top h_t, \quad p_{\text{end}}(t) = \frac{e^{e_t}}{\sum_j e^{e_j}}. \quad (30)$$

Answer span: $\arg \max_{s \leq e} (p_{\text{start}}(s) + p_{\text{end}}(e))$.

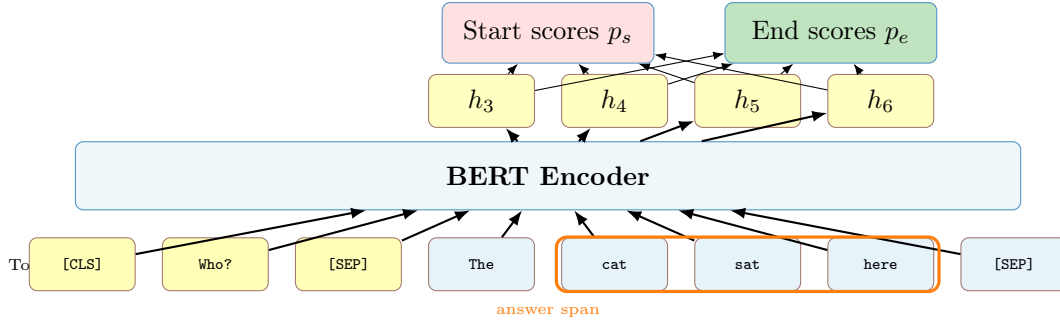


Figure 18: BERT fine-tuned for SQuAD-style QA. Start and end scores are computed for each context token; the highest-probability span is returned.

5.6 BERT Variants and Successors

Model	Params	Key Difference	Notes
BERT-Base	110M	Baseline	12L, 768H, 12A
BERT-Large	340M	Deeper/wider	24L, 1024H, 16A
RoBERTa	125M	No NSP; dynamic masking; longer training	Outperforms BERT on GLUE
ALBERT	12M	Cross-layer param sharing; SOP instead of NSP	Very parameter-efficient
DistilBERT	66M	Knowledge distillation from BERT-Base	~97% performance, 60% size
SpanBERT	110M	Span masking; SBO objective	Better on span tasks
DeBERTa	140M	Disentangled attention; virtual token	State-of-art on many NLU tasks

Table 3: BERT variants and their key innovations.

6 Autoregressive Language Models: GPT Family

6.1 Causal (Decoder-Only) Transformer

GPT models use the *decoder stack only* — no cross-attention, no encoder:

$$p_{\theta}(x_1, \dots, x_T) = \prod_{t=1}^T p_{\theta}(x_t \mid x_1, \dots, x_{t-1}).$$

Each layer is a standard decoder block with a causal mask; the model cannot attend to future tokens. Training objective: *next-token prediction* (causal language modelling, CLM).

CLM Objective

$$\mathcal{L}_{\text{CLM}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(x_t \mid x_1, \dots, x_{t-1}).$$

6.2 Scaling Laws

Kaplan et al. (2020) established that LLM performance scales as a power law in compute C , data D , and parameters N :

$$\mathcal{L} \approx \left(\frac{C_{\min}}{C} \right)^{\alpha_C} + \left(\frac{D_{\min}}{D} \right)^{\alpha_D} + \left(\frac{N_{\min}}{N} \right)^{\alpha_N} + \mathcal{L}_{\infty},$$

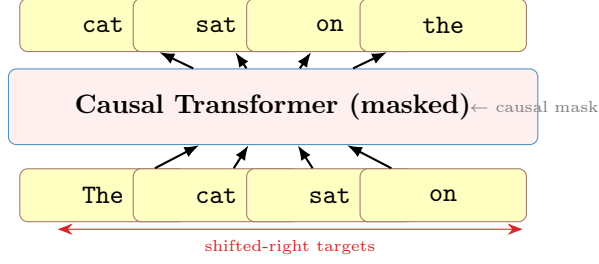


Figure 19: Autoregressive (causal) language model: output at position t predicts token $t + 1$. The causal mask prevents attending to future positions.

where $\alpha_C \approx 0.057$, $\alpha_D \approx 0.095$, $\alpha_N \approx 0.076$. This motivates training the largest model that fits the compute budget on sufficient data.

Model	Year	Params	Tokens trained	Key Feature
GPT-1	2018	117M	1B	First GPT, fine-tune paradigm
GPT-2	2019	1.5B	40B	Zero-shot text generation
GPT-3	2020	175B	300B	Few-shot / in-context learning
Codex	2021	12B	100B code	Code generation (GitHub Copilot)
GPT-3.5 / InstructGPT	2022	175B	300B+	RLHF instruction following
GPT-4	2023	$\sim 1\text{T}$ (est.)	$> 1\text{T}$	Multi-modal, state-of-art NLU

Table 4: GPT model family milestones.

7 Large Language Models (LLMs)

7.1 Architecture Patterns in Modern LLMs

Modern LLMs refine the basic Transformer with several improvements:

Pre-Norm vs. Post-Norm. Original Transformer uses *post-norm* (LN after residual add). Most modern LLMs use *pre-norm* (LN before sub-layer):

$$H^{(l)} = H^{(l-1)} + \text{SubLayer}(\text{LN}(H^{(l-1)})).$$

Pre-norm yields more stable training at large scales.

RMSNorm. Replaces LayerNorm with the simpler:

$$\text{RMSNorm}(h) = \frac{h}{\text{RMS}(h)} \odot \gamma, \quad \text{RMS}(h) = \sqrt{\frac{1}{H} \sum_i h_i^2 + \epsilon}.$$

Used in LLaMA, Mistral, Gemma.

SwiGLU / GeGLU FFN. Replace standard FFN with a gated variant:

$$\text{SwiGLU}(h) = (\text{SiLU}(hW_1) \odot hW_3) W_2,$$

where $\text{SiLU}(x) = x\sigma(x)$. Uses three weight matrices but increases expressive power.

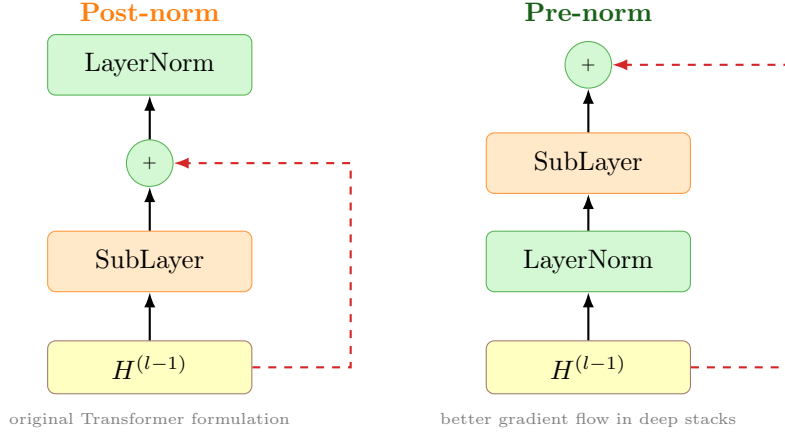


Figure 20: Post-norm applies normalisation after the residual addition; pre-norm applies it before the sub-layer and usually trains more robustly in large LLMs.

Grouped Query Attention (GQA). Standard MHA has h heads each with its own K, V projections. GQA groups the h query heads into g groups sharing K, V , reducing KV-cache memory by factor h/g :

$$\text{head}_i = \text{Attn}(Q_i W_i^Q, K_{\lceil ig/h \rceil} W^K, V_{\lceil ig/h \rceil} W^V).$$

7.2 Instruction Tuning and RLHF

Pre-trained LLMs generate text in the style of training data. To make them helpful assistants, two additional training stages are common:

1. **Supervised Fine-Tuning (SFT):** Fine-tune on curated (instruction, response) pairs.
2. **Reinforcement Learning from Human Feedback (RLHF):**
 - a. Train a *reward model* r_ϕ from human preference comparisons.
 - b. Optimise the LLM policy π_θ with PPO:

$$\mathcal{L}_{\text{RLHF}} = \mathbb{E}[r_\phi(x, y)] - \beta \text{KL}(\pi_\theta \| \pi_{\text{SFT}}).$$

The KL penalty prevents the policy from deviating too far from the SFT baseline and producing degenerate text that scores high on the (imperfect) reward model.

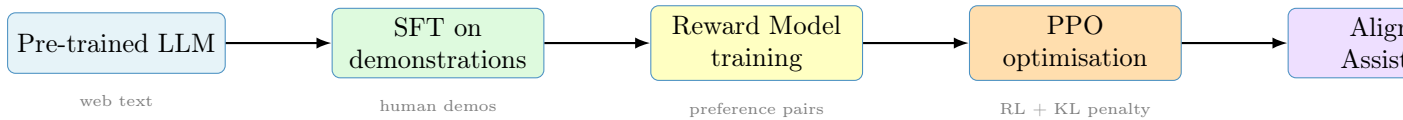


Figure 21: RLHF pipeline (InstructGPT / ChatGPT / Claude). Each stage aligns the model more closely with human intent.

7.3 Key Modern LLMs

7.4 Mixture of Experts (MoE)

Instead of a dense FFN, MoE replaces it with E expert FFNs and a *router*:

$$\text{MoE}(h) = \sum_{i=1}^E g_i(h) \cdot \text{FFN}_i(h),$$

Model	Org	Params	Open?	Key Innovations
LLaMA-2	Meta	7B–70B	Weights	RoPE, GQA (70B), SwiGLU
LLaMA-3	Meta	8B–405B	Weights	GQA all sizes, 128K vocab
Mistral 7B	Mistral	7B	Weights	Sliding window attention, GQA
Mixtral 8×7B	Mistral	46.7B	Weights	Sparse MoE, 12.9B active
Gemma 2	Google	2B–27B	Weights	Alternating local/global attn
Phi-3	Microsoft	3.8B	Weights	Small but highly capable
GPT-4o	OpenAI	~200B	No	Multimodal, voice, vision
Claude 3.5	Anthropic	Unknown	No	Constitutional AI + RLHF
Gemini Ultra	Google	Unknown	No	Multimodal, science tasks

Table 5: Notable modern LLMs (as of 2024–2025).

where $g(h) = \text{TopK}(\text{softmax}(Wh + \text{noise}), k)$ selects k experts (typically $k = 2$). Total parameters scale as $E \cdot d_{ff}$ but compute per token scales as $k \cdot d_{ff}$ — parameter efficiency.

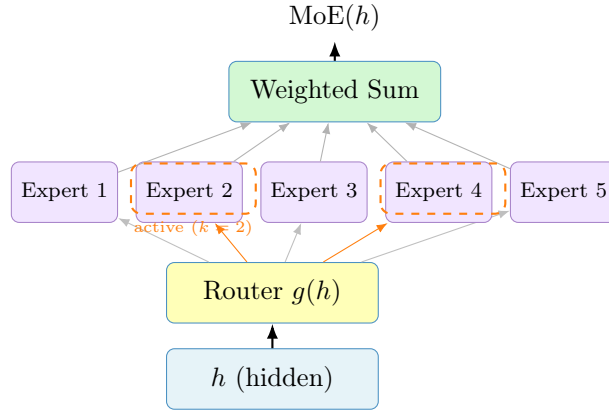


Figure 22: Mixture of Experts layer. The router activates only $k = 2$ of $E = 5$ experts per token, dramatically reducing active compute.

8 Efficient Attention Mechanisms

Standard attention is $O(T^2)$ in memory and compute, limiting context length. Several approaches address this:

8.1 FlashAttention

FlashAttention (Dao et al., 2022) computes the exact same result as standard attention but using *tiled* computation to stay within SRAM (L2 cache), avoiding materialising the full $T \times T$ attention matrix in high-bandwidth memory:

$$O_i = \sum_j \alpha_{ij} v_j, \quad \text{computed in tiles, not all at once.}$$

Reduces memory from $O(T^2)$ to $O(T)$ and achieves 2–4× wall-clock speedup on A100 GPUs.

8.2 Linear Attention

Replace softmax with a kernel function ϕ :

$$\text{LinearAttn}(Q, K, V) = \frac{\phi(Q)(\phi(K)^\top V)}{\phi(Q)\phi(K)^\top \mathbf{1}},$$

using associativity to compute $(\phi(K)^\top V)$ first: $O(Td^2)$ instead of $O(T^2d)$.

8.3 Sparse and Local Attention

Bigbird, Longformer: restrict attention to:

- **Local window**: each token attends to w neighbours.
- **Global tokens**: special tokens (e.g. [CLS]) attend to everything.
- **Random**: a few random pairs.

Reduces to $O(T(w + g + r))$ per layer.

9 Comparing Architectures: A Unified View

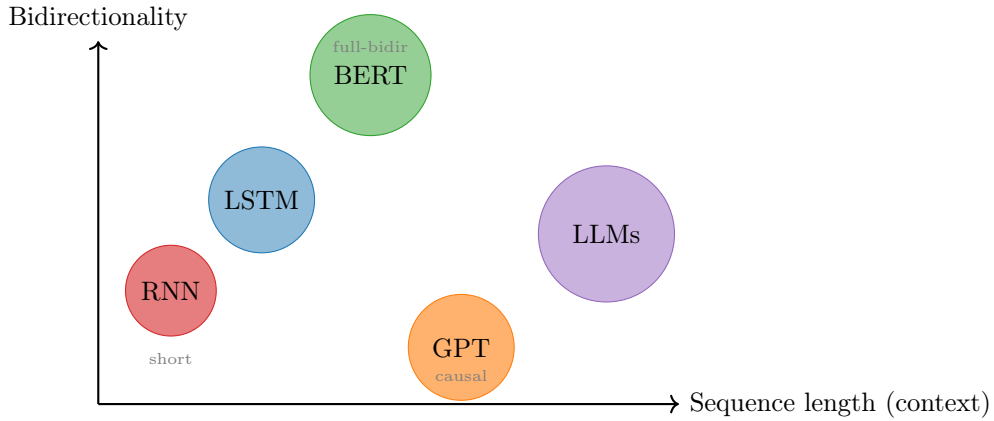


Figure 23: Conceptual map: models placed by context length and degree of bidirectionality.

Aspect	RNN/LSTM	Transformer	BERT	GPT/LLM
Core mechanism	Recurrence + gating	Self-attention	Bidirectional SA	Causal SA
Directionality	Left-to-right (BiRNN for both)	Depends on mask	Fully bidirectional	Left-to-right
Path length	$O(T)$	$O(1)$	$O(1)$	$O(1)$
Parallelism	$O(T)$ sequential	Full	Full	Full
Pre-training	Usually none	Varies	MLM + NSP	CLM
Fine-tuning	Standard	Standard	Add small head	Prompt / PEFT
Memory (train)	$O(T)$	$O(T^2)$	$O(T^2)$	$O(T^2)$

Aspect	RNN/LSTM	Transformer	BERT	GPT/LLM
Typical use	Time series, ASR	Seq2seq, translation	NLU classification	Generation, dialogue

Table 6: Unified comparison of sequential and transformer-based architectures.

10 Tokenisation

All neural language models require mapping raw text to discrete token IDs. Three dominant approaches. In practice, tokenisation is part of a larger text-to-vector pipeline.

10.1 Embedding Lookup and Special Tokens

After tokenisation, each integer ID indexes a learnable row in an embedding matrix $E \in \mathbb{R}^{|\mathcal{V}| \times d}$. For token ID i_t , the embedding at position t is $x_t = E[i_t]$.

Token	Role	Common use
[PAD]	filler to equalise length	ignored by attention mask
[CLS]	pooled summary token	classification in BERT-style models
[SEP]	sequence delimiter	sentence-pair tasks, boundaries
[MASK]	hidden training target	masked language modelling
<bos>/<eos>	begin/end of sequence	decoder generation control

Table 7: Common special tokens used across tokenisation schemes and transformer families.

10.2 Byte-Pair Encoding (BPE)

Start from individual bytes/characters. Iteratively merge the most frequent adjacent pair:

1. Initialise vocabulary $\mathcal{V}$ with all unique characters.
2. Count all adjacent pairs in the training corpus.
3. Merge the most frequent pair $(a, b) \rightarrow ab$; add ab to $\mathcal{V}$.
4. Repeat until $|\mathcal{V}|$ reaches target size.

Used by GPT-2, GPT-3, RoBERTa, LLaMA.

Algorithm 1 Training Byte-Pair Encoding

- 1: Initialise vocabulary $\mathcal{V}$ with base symbols (characters or bytes)
 - 2: Split each training word into base symbols with an end marker
 - 3: **while** $|\mathcal{V}| < V_{\text{target}}$ **do**
 - 4: Count frequencies of all adjacent symbol pairs in the corpus
 - 5: Let $(a^*, b^*) = \arg \max_{(a,b)} \text{count}(a, b)$
 - 6: Merge every occurrence of (a^*, b^*) into a new token a^*b^*
 - 7: Add a^*b^* to $\mathcal{V}$
 - 8: **end while**
 - 9: **return** merge list and final vocabulary
-

Step	Best merge	Illustration
0	—	l o w, l o w e r, n e w e s t, w i d e s t
1	e+s → es	n e w e s t, w i d e s t
2	es+t → est	n e w e s t, w i d e s t
3	l+o → lo	l o w, l o w e r
4	lo+w → low	l o w, l o w e r

Table 8: A small BPE merge history. In practice, real tokenisers learn tens of thousands of merges.

Worked example. Suppose the corpus consists of `low`, `lower`, `newest`, and `widest`. One possible merge sequence is shown in Table 8.

At inference time, BPE greedily applies the learned merges from longest valid subword to shortest.

10.3 WordPiece

Similar to BPE but the merge criterion is likelihood gain rather than raw frequency:

$$\text{score}(a, b) = \frac{p(ab)}{p(a)p(b)}.$$

Used by BERT, DistilBERT, ALBERT.

WordPiece usually marks non-initial subwords with a continuation prefix such as `##`:

$$\text{unaffable} \rightarrow [\text{un}, \text{##aff}, \text{##able}].$$

The prefix makes word boundaries explicit, which simplifies downstream reconstruction.

10.4 Unigram Language Model

Start with a large vocabulary, iteratively remove the token that reduces corpus log-likelihood the least. Used by SentencePiece (T5, mBERT).

In a unigram language model, a tokenisation $z = (z_1, \dots, z_m)$ of a string x is scored by

$$p(x, z) = \prod_{j=1}^m p(z_j), \quad z^* = \arg \max_z \sum_{j=1}^m \log p(z_j).$$

Decoding is typically done with dynamic programming (Viterbi) over all segmentations.

10.5 Comparing Subword Tokenisers and Attention Masks

When batching sequences of unequal length, padding tokens are appended to the shorter examples. An attention mask prevents those positions from influencing real tokens:

$$M_{ij} = \begin{cases} 0, & \text{if token } j \text{ is valid,} \\ -\infty, & \text{if token } j \text{ is padding.} \end{cases}$$

Then attention becomes

$$\text{softmax}\left(\frac{QK^\top + M}{\sqrt{d_k}}\right)V,$$

so padded columns receive probability essentially zero.

11 Training Techniques for LLMs

11.1 Mixed-Precision Training

Store weights in FP32 (or BF16) but compute in FP16, using a *loss scale* to prevent underflow. BF16 (16-bit brain float) is preferred for LLMs: same exponent range as FP32, fewer mantissa bits, numerically more stable than FP16.

11.2 Gradient Checkpointing

Do not store all activations during the forward pass. Re-compute them during the backward pass. Trades compute ($1.33\times$ overhead) for memory reduction proportional to $\sqrt{L}$.

11.3 Parameter-Efficient Fine-Tuning (PEFT)

LoRA (Low-Rank Adaptation). Instead of updating weight $W \in \mathbb{R}^{m \times n}$, learn a low-rank perturbation:

$$W' = W + \Delta W = W + \frac{\alpha}{r}BA,$$

where $A \in \mathbb{R}^{r \times n}$, $B \in \mathbb{R}^{m \times r}$, $r \ll \min(m, n)$. Only A and B are updated; W is frozen. Reduces trainable parameters by $100\text{--}10000\times$.

Prefix Tuning / Prompt Tuning. Prepend trainable “virtual tokens” to the key–value pairs at each layer (prefix tuning) or only at the input (prompt tuning), keeping the LLM weights frozen.

11.4 Data Parallelism, Tensor Parallelism, Pipeline Parallelism

Training a 70B model requires distributing across many GPUs:

- **Data Parallel (DP):** replicate model on k GPUs; each processes a shard of the batch; gradients are averaged (all-reduce).
- **Tensor Parallel (TP):** split weight matrices column/row-wise across GPUs. Requires all-reduce within each layer.
- **Pipeline Parallel (PP):** assign different layers to different GPUs; pass micro-batches through the pipeline.
- **ZeRO:** shard optimizer states, gradients, and parameters across DP ranks to reduce per-GPU memory.

12 Evaluation

12.1 Perplexity

The standard intrinsic metric for language models:

$$\text{PPL}(x_{1:T}) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(x_t \mid x_{<t})\right).$$

Lower perplexity = better. Note: PPL is only comparable across models with the same tokeniser.

Benchmark	Tasks	Measures
GLUE / SuperGLUE	NLI, QA, coref, etc.	NLU breadth
SQuAD 1.1 / 2.0	Extractive QA	F1, EM
MMLU	57-subject MCQ	World knowledge
HumanEval	Code generation	pass@ k
BIG-Bench Hard	Reasoning, logic	Chain-of-thought
HELM	42 scenarios	Multi-dimensional

Table 9: Common LLM evaluation benchmarks.

12.2 Benchmark Suites

13 Conclusion

This document traces a coherent conceptual path:

1. **RNN/LSTM/GRU**: recurrence enables sequence modelling but imposes sequential computation and $O(T)$ dependency paths.
2. **Self-Attention**: connects all positions in $O(1)$ steps with full parallelism. Scaled dot-product attention + multi-head projection is the building block.
3. **Transformer**: self-attention + FFN + residual + layer-norm, stacked L times. Encoder-decoder for seq2seq; encoder-only for understanding; decoder-only for generation.
4. **BERT**: encoder-only, pre-trained with MLM + NSP. Bidirectional context. Fine-tuned with task-specific heads on a wide array of NLU benchmarks.
5. **GPT and LLMs**: decoder-only, pre-trained with CLM at massive scale. Instruction-tuned with SFT + RLHF. Exhibit in-context learning, emergent capabilities, and general-purpose reasoning.
6. **Modern improvements**: RMSNorm, RoPE, GQA, SwiGLU FFN, FlashAttention, MoE, LoRA — each addresses a specific bottleneck in the original Transformer.

Understanding this progression — from the basic recurrence equation to the architectural choices in a 400B-parameter LLM — provides a principled foundation for reasoning about any neural sequence model, past or future.

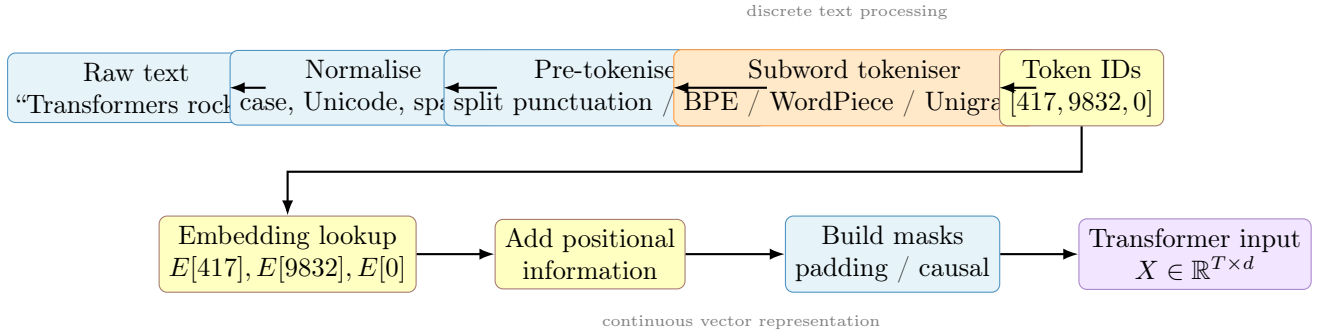


Figure 24: Text-to-vector pipeline used by modern transformer systems. Tokenisation chooses discrete units; embedding lookup converts IDs into dense vectors consumable by neural layers.

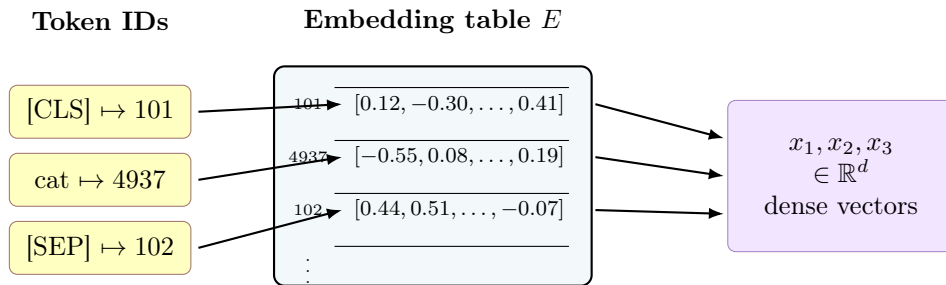


Figure 25: Embedding lookup maps each token ID to a learnable dense vector. These vectors are later combined with positional information and masks before entering the transformer.

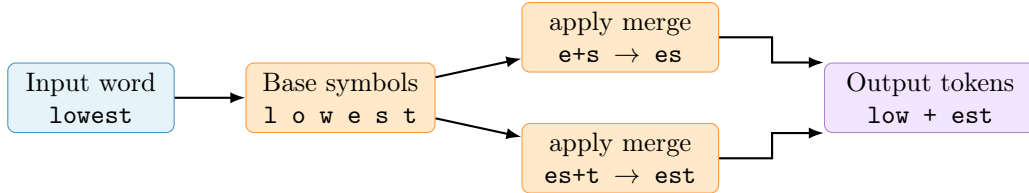


Figure 26: Inference-time BPE tokenisation can be viewed as repeated application of previously learned merges.

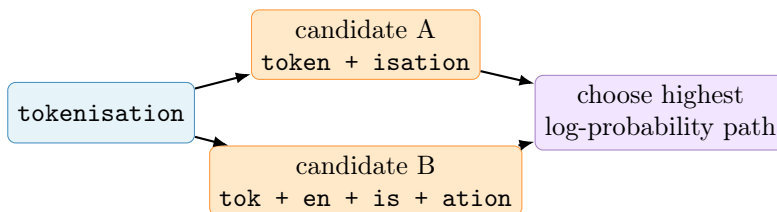


Figure 27: Unigram tokenisation chooses the most probable segmentation under a learned token inventory.

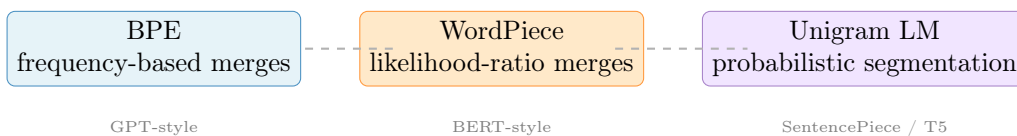


Figure 28: Three common subword tokenisation families. All convert text into variable-length sequences of learned discrete units, but they differ in how the vocabulary is trained.

References

- [1] Vaswani, A., et al. (2017). *Attention Is All You Need*. NeurIPS.
- [2] Devlin, J., et al. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory*. Neural Computation, 9(8).
- [4] Cho, K., et al. (2014). *Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation*. EMNLP.
- [5] Bahdanau, D., et al. (2015). *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR.
- [6] Radford, A., et al. (2018). *Improving Language Understanding by Generative Pre-Training*. OpenAI Blog.
- [7] Brown, T., et al. (2020). *Language Models are Few-Shot Learners*. NeurIPS.
- [8] Kaplan, J., et al. (2020). *Scaling Laws for Neural Language Models*. arXiv:2001.08361.
- [9] Ouyang, L., et al. (2022). *Training Language Models to Follow Instructions with Human Feedback*. NeurIPS.
- [10] Touvron, H., et al. (2023). *LLaMA 2: Open Foundation and Fine-Tuned Chat Models*. arXiv:2307.09288.
- [11] Su, J., et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864.
- [12] Dao, T., et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS.
- [13] Hu, E., et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR.
- [14] Ainslie, J., et al. (2023). *GQA: Training Generalised Multi-Query Transformer Models from Multi-Head Checkpoints*. EMNLP.
- [15] Liu, Y., et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692.
- [16] Lan, Z., et al. (2020). *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations*. ICLR.